Formalización de Quadtrees y algunas de sus Invariantes

Ana Dhamar Cabrera Sánchez Juan Luis Peña Mata

Junio 15, 2026.

1. Introducción

Las imágenes forman parte fundamental del conocimiento que adquiere el ser humano a lo largo de su existencia, pues son un método de aprendizaje y comunicación que nos ayuda a comprender de mejor manera nuestro entorno. Con la evolución del mundo digital, surgió la necesidad de integrar y procesar esta información visual de manera computacional.

Para llevar a cabo este proceso, las imágenes suelen ser representadas mediante dos tipos de archivos: el primero, es conocido como *raster graphic* o *bit map* y es la representación de un arreglo de píxeles; mientras que el segundo es conocido como *archivo vectorial* y contiene ecuaciones especificas que tienen la capacidad de dibujar la imagen almacenada.

Si bien ambas representaciones poseen ventajas y desventajas particulares respecto a la resolución y escala de la imagen que representan, ambos tipos de archivos tienen la necesidad de contar con estructuras de datos eficientes para almacenar, manipular y proyectar la información visual de manera óptima.

Enfocándonos momentáneamente en las etapas de almacenamiento y proyección, destaca una familia de estructuras conocida como *quadtrees*, cuya traducción al español suele ser árboles cuaternarios [1] . Estos árboles están diseñados específicamente para subdividir en regiones regulares el espacio bidimensional y recuperar eficientemente la información que almacenan, siendo aplicables tanto para archivos vectoriales como para archivos de pixeles.

En este trabajo nos centraremos en los quadtrees destinados a la representación de imágenes con pixeles, puesto que en la actualidad existen múltiples sistemas de software que utilizan estas variantes en áreas como la medicina, la aeronáutica y la automatización en la detección de objetos. A partir de sus aplicaciones, resulta evidente que muchas de las disciplinas donde los quadtrees son requeridos forman parte del desarrollo de software crítico.

Derivado de lo anterior, existe una inherente necesidad de formalizar el comportamiento de esta estructura de datos para garantizar la ausencia de fallos críticos o pérdida de información, llevándonos a la pregunta base de esta investigación: *¿Qué invariantes deben mantenerse en los árboles cuaternarios destinados a interpretar imágenes digitalmente y cómo se formalizan?*

En el contenido de este trabajo, se propone verificar formalmente el comportamiento principal de la estructura de datos utilizando el asistente de pruebas Rocq, fundamentando la investigación y las decisiones de diseño en la correspondencia de Curry-Howard, los principios de la geometría computacional discreta y la teoría de tipos dependientes.

[1] En este trabajo utilizaremos la convención estándar *quadtree*, que fue dada por Finkel y Bentley en 1974 [1]. Sin embargo, es necesario destacar que los *quadtrees* pensados para la representación de imágenes, fueron introducidos por Klinger y Dyer en su artículo *"Experiments on Picture Representation Using Regular Decomposition"* en 1976 [2], bajo el nombre de *Q-tree*.

2. Marco Teórico

Para demostrar que la subdivisión realizada por el algoritmo de los quadtrees preserva estrictamente la identidad geométrica de la imagen original y garantiza, a su vez, una complejidad espacial óptima es necesario conocer como se asocia la representación de imágenes, el razonamiento geométrico automatizado y los sistemas de tipos.

Por ello, se desglosarán los conceptos más relevantes en las siguientes secciones.

2.1. Quadtrees

Un *quadtree* es una estructura de datos jerárquica para imágenes, en la cual la información visual se divide recursivamente en cuatro regiones hasta encontrar un patrón lo suficientemente simple como para ser almacenado en una estructura más sencilla [3].

En esencia, el nodo raíz representa la imagen completa. Al subdividirse en cuatro regiones, cada nodo interno deja de contener la información visual directa y se convierte en un nodo gris, y siguiendo recursivamente la subdivisión, todos los nodos padre cumplen la propiedad de ser nodos grises, delegando exclusivamente a los nodos hoja la contención de las regiones simples de la imagen original.

Es importante destacar que los *quadtrees* son una familia de árboles cuyo algoritmo se basa en las especificaciones que hemos descrito en los párrafos anteriores [3]. Dicha familia se compone de diversas variantes que, por naturaleza del algoritmo, dependen estrictamente de la posición de los cuadrantes que conforman la imagen que almacenan, por lo que cada nodo hijo corresponde a una región espacial específica y ordenada, a la que le corresponde una de las siguientes etiquetas: noroeste (NW), noreste (NE), suroeste (SW) ó sureste (SE).

Para almacenar de manera correcta las imágenes, es requisito indispensable cuidar que la imagen original tenga dimensiones $2^n \times 2^n$, pues esto garantiza que tras cada división, siempre se obtendran subregiones exactas de tamaño $2^{n-1} \times 2^{n-1}$, con $n \in \mathbb{N}$.

2.2. Procesamiento de Imágenes

Las imágenes que se procesan en el software crítico, suelen estar fuertemente ligadas a los archivos de píxeles [4,5] , esto se debe a que la representación indicada es capaz de preservar más información y detalle de una imagen que los archivos vectoriales.

Dicha representación es conocida como *raster graphic* o *bit map*. En este modelo, el pixel se define como la unidad visual programable más pequeña y la representación de la imagen depende directamente de una matriz de pixeles, donde, a mayor tamaño del pixel, se obtiene una menor resolución, por lo que mejorar la calidad visual exige incrementar la cantidad de pixeles. De ello se tiene que las variantes de los quadtrees que se crearon para este tipo de archivos deben ser óptimas en las complejidades de tiempo y espacio.

Por otro lado, la representación mediante vectores, utiliza fórmulas matemáticas para definir imágenes. En este enfoque se necesita una cantidad de información mínima para representar la imagen, por lo que los archivos suelen ser más pequeños respecto a la representación anterior; adicionalmente, tienen la facilidad de ser escalables pues los ajustes se realizan modificando las ecuaciones, lo que permite reescalar la imagen sin pérdida de calidad. Su desventaja principal es que suele limitarse a formas geométricas simples, como líneas, círculos y polígonos regulares, sin embargo, siguen siendo presentados y modificados para tomar decisiones dentro del software crítico [6].

2.3. Verificación Formal y Asistentes de Prueba

En el desarrollo de software crítico, resulta indispensable la verificación de las decisiones de diseño que se ejecutan, no basta con realizar pruebas unitarias, debemos garantizar que el programa no contenga defectos lógicos introducidos durante su desarrollo. Para llevar a cabo esta tarea, es correcto recurrir a la verificación mediante asistentes de prueba, herramientas híbridas que automatizan los aspectos mecánicos de la demostración lógica bajo la guía de una persona que tenga el conocimiento de programación requerido [7].

Para la formalización de los quadtrees, utilizaremos Rocq (anteriormente conocido como Coq), un asistente de prueba cuyo núcleo se fundamenta en el Cálculo de Construcciones [8], por ende, este formalismo se basa en la correspondencia de Curry-Howard, la cual establece que las proposiciones lógicas son equivalentes a los tipos, y las demostraciones a las funciones que habitan dichos tipos; por lo que, en el paradigma donde se desarrolla Rocq, las demostraciones son en sí mismas programas funcionales [8].

Una de las características más importantes que provee el Cálculo de Construcciones, es la incorporación de la teoría de tipos dependientes. Lo que diferencia a esta teoría de otros sistemas de tipificado es que permite refinar una definición de tipo sobre un valor específico, por lo cual, no muestra una separación clara entre tipos y programas [9]. Incorporar este refinamiento desde el tiempo de compilación, le permite al compilador exigir la verificación de que un valor cumple con las especificaciones que se han asociado según su tipo y eso, es justamente lo que nos permitirá formalizar con mayor facilidad la representación de imágenes como se abordará en la siguiente sección.

2.4. Demostraciones Automatizadas en Geometría

Antes de adentrarnos con el uso de un asistente de pruebas como método para formalizar la estructura de datos indicada, es indispensable recurrir a lo que se ha desarrollado hasta el momento sobre Deducción Automatizada en Geometría para comprender los retos que se presentan al querer utilizar un asistente de pruebas que verifique propiedades sobre objetos geométricos, cómo lo son las imágenes.

Para ello, tomaremos como base las ideas plasmadas en el articulo *"Proof and computation in geometry"*, escrito por Michael Beeson [10]; donde el autor analiza la relación que existe entre el álgebra, la geometría, la computación y las demostraciones formales.

A través del diagrama ilustrado en la Figura 1, Beeson contrasta la diferencia entre las demostraciones realizadas tomando como base la geometría de los objetos y las demostraciones que se modelan en un dominio álgebraico. Argumentando que la diferencia principal radica en que las demostraciones que usan el método algebraico se comportan como un predicado en computación, limitándose a confirmar o negar que un resultado es verdadero; mientras que las pruebas geométricas nos dan información de *por qué* lo es.

Sin embargo, la obtención de pruebas geométricas, ha demostrado ser una tarea sumamente difícil en la computación, razón por la cual recurre a la analogía del dragón para señalizarlo, remarcando que este tipo de verificación se considera un territorio inexplorado y posiblemente peligroso para demostraciones automatizadas.

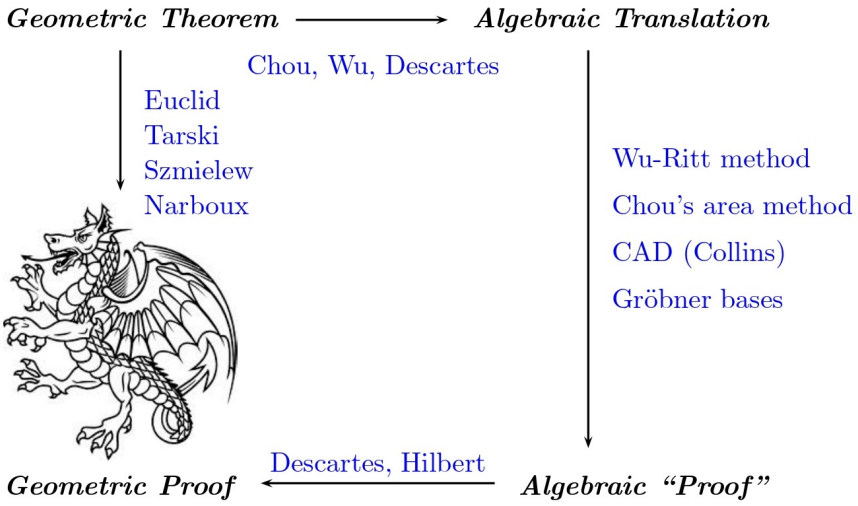


Figura 1: "Commutative Diagram" presentado por Michael Beeson [10] (Sección 1.1, pág. 2)

En el presente trabajo, tomaremos la estrategia estándar para la verificación en programas (marcada por la ruta que rodea al dragón en la Figura 1) por ello, en lugar de hablar de la topología de las imágenes, se hará una traducción del problema geométrico a su dominio algebraico, donde se le dará solución a las demostraciones de nuestro interés, y remarcaremos como la traducción inversa (*back-translation*) para proyectar el resultado de vuelta al dominio original es inmediata según nuestra implementación.

3. Formalización de Imágenes

Para verificar el comportamiento e invariantes de los quadtrees, el primer paso consiste en establecer la interpretación geométrica del espacio donde se plasmarán las imágenes. En esta sección, notaremos que la complejidad que tendrán las demostraciones necesarias para nuestra estructura de datos puede complicarse según nuestra elección para formalizar las imágenes.

3.1. Intuición Seguida de Limitaciones

Siguiendo el pensamiento al cual suele asociarse el paradigma de programación funcional, podemos pensar que utilizar listas (específicamente, listas de listas) para modelar matrices que simulen imágenes puede ser la solución ideal.

El razonamiento de la interpretación suena simple: cada sublista representa una línea horizontal de la imagen, y cada elemento individual contiene la información de un píxel específico. Sin embargo, al querer automatizar las demostraciones utilizando este enfoque, la solución intuitiva genera una cantidad excesiva de restricciones que dificultan la formalización de nuestra estructura de datos principal.

Lo anterior deriva de intentar que Rocq se convenza de que, al subdividir recursivamente una lista bidimensional, las matrices resultantes preservan las propiedades estructurales exigidas por la familia de los quadtrees. Garantizar invariantes como las dimensiones acotadas por potencias de la forma 2^n, la conservación absoluta del contenido la imagen y la exactitud de las particiones, exige una cantidad desmesurada de lemas auxiliares enfocados puramente a la validación de índices y fronteras de la matriz.

Es en este punto donde la teoría de tipos dependientes ofrece una solución matemática más elegante y robusta. Sustituyendo el uso de estructuras de datos auxiliares para la representación de imágenes, recurriremos a la estrategia estándar presentada en la Sección 2.3, y, utilizando restricciones algebraicas que puedan modelarse a través de tipos dependientes, conseguiremos una traducción adecuada para modelar imágenes concretas que cumplan los requisitos necesarios para poder ser almacenadas y procesadas correctamente en las variantes de quadtrees destinadas al uso de píxeles.

3.2. Imágenes y Tipos Dependientes

Para representar las imágenes de manera discreta y trasladar las precondiciones de los quadtrees directamente al dominio algebraico, es necesario visualizar un espacio geométrico donde se incrusten nuestras imágenes. Para ello, proponemos que el posicionamiento geométrico se realice en el primer cuadrante del plano cartesiano, de manera que cada píxel sea asociado a la coordenada de su esquina inferior izquierda; en contraste con la representación de matrices, es importante mencionar que el píxel colocado en la coordenada $(0, 0)$ corresponde a la esquina inferior izquierda de la imagen, tal y como se muestra en la Figura 2.

Adicionalmente, en la Figura 3 podemos notar cómo proyectar las imágenes sobre un espacio cartesiano permite asignar las etiquetas: Noroeste (NW), Noreste (NE), Suroeste (SW) y Sureste (SE) de manera precisa a las regiones obtenidas tras dividir el espacio de manera homogénea.

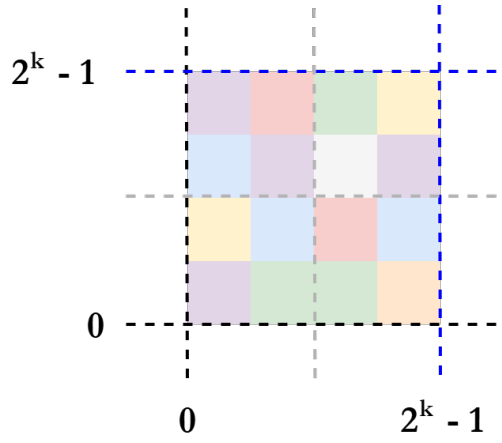


Figura 2: Representación de la imagen en el primer cuadrante del plano.

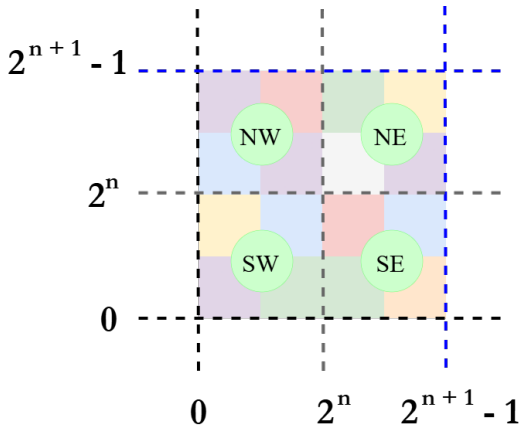


Figura 3: División de Cuadrantes de la imagen presentada en la Figura 2.

Establecida esta representación espacial, el primer paso para la formalización mediante tipos dependientes consiste en definir la unidad visual fundamental: el píxel.

Definimos un píxel como un valor booleano, donde los posibles valores asociados: verdadero y falso, actuarán como simuladores de color [2] :

```
1 Definition pixel := bool.
```

[2]La justificación sobre esta decisión se abordará con precisión en la sección Sección 4.1

Considerando esta definición y siguiendo la interpretación mostrada en la Figura 2, es posible definir una imagen como una función dependiente enfocada en la traducción del espacio geométrico al dominio algebraico:

```
Definition imagen (n : nat) :=
  ∀ (x y : nat), x < 2^n → y < 2^n → pixel.
```

Notemos como la función asocia dos números naturales, que simulan ser coordenadas de la forma (x, y) dentro de un plano cartesiano, y exige pruebas formales de que ambos valores son estrictamente menores que 2^n, lo cual garantiza que la imagen tiene proporciones exactas y un tamaño acotado de $2^n \times 2^n$. Más aún, la función devuelve el píxel correspondiente a la coordenada formada por (x, y) únicamente si la coordenada está dentro de los límites establecidos para la imagen.

Para indicar qué valores pueden tomar las coordenadas dada la proporción de una imagen, es indispensable verificar cómo se comportan sus límites y cuadrantes al trabajar con distintas resoluciones. Para ello, es necesario demostrar dos propiedades, que hemos definido como lemas en Rocq.

3.2.1. Cuadrante → Imagen

El primer lema establece una propiedad inherente de la subdivisión de imágenes: si una coordenada pertenece a un cuadrante delimitado por 2^n, lógicamente se encuentra dentro de los límites de la imagen que lo contiene, delimitada por 2^{n+1}.

```
Lemma xy_in_img : ∀ v n, v < 2^n → v < 2^(S n).
Proof.
  intros. simpl. lia.
Qed.
```

Notemos que la demostración de este lema se resuelve de manera simple, esto deriva del modelo algebraico con el que hemos representado las imágenes, el cual le permite a la táctica de aritmética lineal, `lia`, resolver la meta $v < 2^n \to v < 2^{n+1}$.

3.2.2. Lema de traslación

Adicional al lema anterior, necesitamos establecer qué sucede cuando las coordenadas asociadas a un píxel se desplazan, especialmente cuando se desplazan hacia otros cuadrantes, por lo cual necesitamos el lema de traslación enunciado como sigue:

```
Lemma xy_in_img_translacion : ∀ v n, v < 2^n → v + 2^n < 2^(S n).
Proof.
  intros. simpl. lia.
Qed.
```

Este lema indica que si a una coordenada válida en una imagen de tamaño $2^n \times 2^n$ se le suma la longitud del cuadrante (2^n), el límite resultante permanece estrictamente dentro de las dimensiones de una imagen del doble de tamaño (2^{n+1}). Esta propiedad asegura que para cada uno de los píxeles de una sub-imagen, que se define usando la función dependiente presentada con anterioridad, los desplazamientos en el plano son matemáticamente seguros dentro de la imagen global, previniendo desbordamientos espaciales.

3.3. Cuadrantes desde la Traducción Algebraica

Una vez establecida la traducción al dominio algebráico de las imágenes, necesitamos formalizar la partición geométrica del plano en regiones homogeneas, es decir, formalizar adecuadamente los cuadrantes NW, NE, SW, SE. Para ello, consideraremos una imagen de tamaño 2^{n+1}, y a partir de ella, modelaremos subimágenes de tamaño 2^n que representen las características de cada cuadrante, tal y como se ilustró en la Figura 3.

Siguiendo la especificación anterior, es necesario calcular la traslación necesaria a partir de las coordenadas (x, y) de nuestra subimagen, exigiendo, cómo se indica en la definición, las pruebas $H_x : x < 2^n$ y $H_y : y < 2^n$.

3.3.1. Cuadrante NW

La región Noroeste (NW) se caracteriza por mantener los valores de sus coordenadas (x, y) en los rangos: $x \in [0, 2^n - 1]$, $y \in [2^n, 2^{n+1} - 1]$. Por lo tanto, la función que necesitamos para definir la subimagen que representa a este cuadrante requiere de todas las coordenadas locales en las que y se trasladó 2^n unidades en el plano, mientras que x mantuvo su valor original.

Para mostrar que estos valores se encontrarán dentro de los límites de nuestra imagen global, recurrimos a los lemas desarrollados en la sección anterior, de manera que el cuadrante NW se define como sigue:

```
Definition cuadrante_NW {n : nat} (img : imagen (S n)) : imagen n :=
  fun x y Hx Hy ⇒
    img x (y + 2^n) (xy_in_img x n Hx) (xy_in_img_translacion y n Hy).
```

3.3.2. Cuadrante NE

Para la región Noreste (NE), la ubicación correspondiente en el plano exige que tanto x como y pertenezcan al rango $[2^n, 2^{n+1} - 1]$. De ello se sigue que la definición formal de este cuadrante traslada 2^n unidades los valores locales de x, y.

Aplicando el lema de traslación en ambos ejes podemos garantizar que los valores tomados por las coordenadas globales se mantienen dentro de los límites de imagen original:

```
Definition cuadrante_NE {n : nat} (img : imagen (S n)) : imagen n :=
  fun x y Hx Hy ⇒
    img (x + 2^n) (y + 2^n) (xy_in_img_translacion x n Hx)
                            (xy_in_img_translacion y n Hy).
```

3.3.3. Cuadrante SW

La región Suroeste (SW) de nuestra imagen se ubica en el origen de nuestro plano cartesiano. Notemos entonces que las coordenadas x, y varían únicamente de 0 a $2^n - 1$.

Lo anterior deriva en que las coordenadas globales y las coordenadas locales son exactamente las mismas (i. e., no se requiere ningún desplazamiento para definir las coordenadas que corresponden a este cuadrante):

```
Definition cuadrante_SW {n : nat} (img : imagen (S n)) : imagen n :=
  fun x y Hx Hy ⇒
    img x y (xy_in_img x n Hx) (xy_in_img y n Hy).
```

3.3.4. Cuadrante SE

Para la región Sureste (SE) de nuestra imagen, tenemos un comportamiento simétrico a la región NW, pues x varía desde 2^n hasta $2^{n+1} - 1$, mientras que y se mantiene fija.

Es por ello que la definición requerida para este cuadrante se obtiene con la siguiente especificación:

```
Definition cuadrante_SE {n : nat} (img : imagen (S n)) : imagen n :=
  fun x y Hx Hy ⇒
    img (x + 2^n) y (xy_in_img_translacion x n Hx) (xy_in_img y n Hy).
```

3.4. Mapeo de Coordenadas y Ensamblaje de Cuadrantes

El objetivo central para la reconstrucción de imágenes es definir una función de ensamblaje. Para crear una imagen completa de tamaño 2^{n+1} a partir de cuatro subimágenes (cuadrantes) de tamaño 2^n, requerimos determinar con precisión en qué cuadrante local recaerá cada una de las coordenadas de la nueva imagen; a estas nuevas coordenadas las denominaremos *coordenadas globales*.

3.4.1. Lema de localización

Consideremos, sin pérdida de generalidad, que el valor de una coordenada global x se encuentra en la segunda mitad del plano (i.e., en el lado derecho de la imagen). Sabemos entonces que $x \geq 2^n$, por lo que debemos buscar el píxel que la representa en los cuadrantes derechos (SE o NE). Sin embargo, todos los subcuadrantes son, por definición, imágenes independientes cuyas coordenadas se encuentran estrictamente dentro del rango $[0, 2^n - 1]$.

Para encontrar correctamente la coordenada global en el interior de un cuadrante, es necesario sustraer 2^n unidades al valor de x. Naturalmente, Rocq solicitará una prueba formal para asegurar que la coordenada asignada, cuyo valor será $x - 2^n$, jamás excederá los límites del cuadrante destino. Dicha proposición se demuestra en el siguiente lema:

```
Lemma localiza_xy_desde_2n :
    ∀ v n, ~ (v < 2^n) → v < 2^(S n) → v - 2^n < 2^n.
Proof.
  intros.
  simpl in H0.
  lia.
Qed.
```

El lema establece que si v no es menor que 2^n, lo cual es equivalente a $v \geq 2^n$, y además se cumple el límite global $v < 2^n + 2^n$, la diferencia $v - 2^n$ siempre resultará estrictamente menor que 2^n. Notemos nuevamente que esta afirmación es resuelta por la táctica `lia` de manera inmediata, confirmando que el dominio algebraico es un entorno robusto para validar traslaciones espaciales.

3.4.2. La Función de Ensamblaje

Apoyándonos en el lema de localización, definimos la operación `ensamblar`, la cual recibe cuatro subimágenes de tamaño 2^n y devuelve una imagen global de proporciones 2^{n+1}.

Para determinar a qué subimagen pertenece una coordenada global (x, y), necesitamos evaluar tanto x como y respecto al punto medio de la imagen, 2^n.

Para seguir con la idea anterior, recurrimos a la función `lt_dec`, la cual devuelve `left H` si $a < b$ ó `right H` si $a \geq b$, para cualesquiera valores a, b. La combinación de los resultados aplicados a las coordenadas (x, y), generan cuatro escenarios:

```
Definition ensamblar {n : nat}
  (nw ne sw se : imagen n) : imagen (S n) :=
  fun x y Hx Hy ⇒
    match lt_dec x (2^n), lt_dec y (2^n) with
    (* Caso 1: x < 2^n, y >= 2^n
       estamos en el Cuadrante NW *)
    | left Hx_lt, right Hy_ge ⇒
        nw x (y - 2^n) Hx_lt (localiza_xy_desde_2n y n Hy_ge Hy)
    (* Caso 2: x >= 2^n, y >= 2^n
       estamos en el Cuadrante NE *)
    | right Hx_ge, right Hy_ge ⇒
        ne (x - 2^n) (y - 2^n) (localiza_xy_desde_2n x n Hx_ge Hx)
                               (localiza_xy_desde_2n y n Hy_ge Hy)
    (* Caso 3: x < 2^n, y < 2^n,
       estamos en el Cuadrante SW *)
    | left Hx_lt, left Hy_lt ⇒
        sw x y Hx_lt Hy_lt
    (* Caso 4: x >= 2^n, y < 2^n
       estamos en el Cuadrante SE *)
    | right Hx_ge, left Hy_lt ⇒
        se (x - 2^n) y (localiza_xy_desde_2n x n Hx_ge Hx) Hy_lt
    end.
```

El bloque `match` nos brinda la flexibilidad de resolver cada uno de los casos posibles. Si ambas coordenadas son menores al punto medio, nos encontramos anclados al origen en el Cuadrante SW. Si alguna o ambas coordenadas exceden este umbral, se sustraen 2^n unidades en los ejes correspondientes y se usa el lema de localización para obtener las coordenadas locales dentro de cada cuadrante.

3.4.3. Lema Mapeo de Pixeles

Para garantizar que dividir y ensamblar cuadrantes produce la misma imagen, primero debemos resolver un paso intermedio, al utilizar funciones dependientes, cada llamada requiere no solo los valores numéricos de las coordenadas, también las pruebas formales de sus límites. Para la manipulación de imágenes, establecemos un principio de irrelevancia de pruebas: dos evaluaciones sobre una imagen devuelven el mismo píxel si sus coordenadas numéricas son idénticas ($x_1 = x_2$, $y_1 = y_2$), siendo irrelevante el camino lógico mediante el cual se demostró que cumplían las cotas.

```
Lemma pixel_eq : ∀ n (img : imagen n) x1 y1 x2 y2
  (Hx1 : x1 < 2^n) (Hy1 : y1 < 2^n)
  (Hx2 : x2 < 2^n) (Hy2 : y2 < 2^n),
  x1 = x2 → y1 = y2 →
  img x1 y1 Hx1 Hy1 = img x2 y2 Hx2 Hy2.
Proof.
  intros.
  subst.
  f_equal.
  - apply le_unique.
  - apply le_unique.
Qed.
```

Al aplicar la sustitución de valores asociados a las coordenadas en ambos lados de la igual-

dad, basta utilizar `f_equal` para aislar las demostraciones lógicas. La táctica `le_unique` es aplicada para mostrar que cualquier demostración de desigualdad sobre números naturales es válida, pues, por el principio de buen orden aplicado en los números naturales, sabemos que cualquier prueba debe ser equivalente.

3.4.4. Teorema de Ensamblaje

Con la igualdad de píxeles fundamentada, podemos presentar el resultado central de esta sección: la demostración de que dividir una imagen en sus cuatro cuadrantes y reconstruirla usando la función `ensamblar` preserva de manera total la información de la imagen original.

```
Theorem teorema_ensamblaje : ∀ {n} (img : imagen (S n)),
    ensamblar (cuadrante_NW img) (cuadrante_NE img)
              (cuadrante_SW img) (cuadrante_SE img) = img.
Proof.
  intros.
  extensionality x, y, Hx, Hy.
  unfold ensamblar.

  destruct (lt_dec x (2^n)) as [Hx_lt | Hx_ge];
  destruct (lt_dec y (2^n)) as [Hy_lt | Hy_ge].
  - unfold cuadrante_SW.
    apply pixel_eq; reflexivity.
  - unfold cuadrante_NW.
    apply pixel_eq.
    + reflexivity.
    + lia.
  - unfold cuadrante_SE.
    apply pixel_eq; lia.
  - unfold cuadrante_NE.
    apply pixel_eq; lia.
Qed.
```

Dado que modelamos la imagen como una función, es necesario utilizar extensionalidad funcional [11] para demostrarlo. Procedemos por casos según los escenarios descritos por `lt_dec`:

- **Caso 1: SW** ($x < 2^n, y < 2^n$). Las coordenadas globales corresponden exactamente a las locales. No ocurre ninguna traslación, por lo cual basta aplicar reflexividad para cerrar la submeta.
- **Caso 2: NW** ($x < 2^n, y \geq 2^n$). La función `ensamblar` extrae el píxel restando 2^n a la coordenada global y. Por definición, el cuadrante proyecta esta consulta sumándole el mismo valor. Lo cual genera la igualdad [3] : $(y - 2^n) + 2^n = y$.
- **Caso 3: SE** ($x \geq 2^n, y < 2^n$). Analogamente, la función `ensamblar` extrae el píxel restando 2^n a la coordenada x, mientras que la definición del cuadrante lo compensa sumando la misma constante. Por lo que se obtiene la igualdad [4] : $(x - 2^n) + 2^n = x$.
- **Caso 4: NE** ($x \geq 2^n, y \geq 2^n$). Ocurre un desplazamiento en ambas coordenadas. En este caso, x tiene el mismo comportamiento que en el caso 3 y y tiene el mismo comportamiento que en el caso 2, por lo cual, las desigualdades son correctas.

[3] Como $y \in \mathbb{N}; y \geq 2^n$, la igualdad es segura.
[4] Como $x \in \mathbb{N}; x \geq 2^n$, la igualdad es segura.

4. Region Quadtree

Como hemos desarrollado en la Sección 2.1, los quadtrees constituyen una extensa familia de árboles diseñada para la subdivisión espacial. Dentro de esta clasificación, una de las variantes donde el algoritmo general se ve reflejado de manera directa correspondiendo a la representación de imágenes lleva el nombre de *Region Quadtree*. Es precisamente en esta variante en la que nos centraremos a continuación, con el objetivo de formalizar su construcción y demostrar los invariantes que produce su comportamiento.

4.1. Definición en Rocq

El propósito principal del Region Quadtree es comprimir la información espacial agrupando áreas contiguas que comparten un mismo valor visual [3]. En consecuencia a lo anterior, resulta suficiente modelar las imágenes utilizando únicamente dos colores (tal y como se definió el píxel en la Sección 3.2) para lograr la formalización de la estructura de datos y analizar sus propiedades.

Basándonos en este razonamiento, el comportamiento del RQT se traduce al asistente de prueba mediante un tipo inductivo que garantiza su construcción:

```
Inductive region_qt : Type :=
  | Leaf : pixel → region_qt
  | Node : region_qt → region_qt → region_qt → region_qt → region_qt.
```

Cualquier región del plano que sea completamente homogénea se modela como una hoja en el árbol usando el constructor `Leaf`. Este constructor necesita del píxel como argumento para recordar el color asociado a la región que se representa en el caso base.

Cuando se presenta una región no uniforme de color, la definición exige que el plano sea dividido para continuar recursivamente sobre los cuadrantes, y es justo este comportamiento el que modela a un nodo intermedio. El constructor `Node` representa los nodos grises de la subdivisión y requiere de cuatro argumentos, los cuales, se asocian de manera ordenada a los cuadrantes: NW, NE, SW, SE.

4.2. Construcción RQT

Es inmediato notar que la construcción de un Region Quadtree (RQT) tiene la necesidad de determinar cuando una sección de color está compuesta por un color único, y cuando se compone por una variación de colores. La validación necesaria se realiza mediante la función auxiliar `es_homogenea`, la cual se desenvuelve de manera recursiva sobre el exponente n que dicta las proporciones de la imagen:

```
Fixpoint es_homogenea (n : nat) : imagen n → option pixel :=
  match n with
    | 0 ⇒
        fun img ⇒
          Some (img 0 0 (Nat.lt_0_1) (Nat.lt_0_1))
    | S n' ⇒
        fun img ⇒
          match es_homogenea n' (cuadrante_NW img),
                es_homogenea n' (cuadrante_NE img),
                es_homogenea n' (cuadrante_SW img),
                es_homogenea n' (cuadrante_SE img)
          with
```

```
            | Some c_nw, Some c_ne, Some c_sw, Some c_se ⇒
            match c_nw, c_ne, c_sw, c_se with
              | true, true, true, true ⇒ Some true
              | false, false, false, false ⇒ Some false
              | _, _, _, _ ⇒ None
            end
            | _, _, _, _ ⇒ None
          end
  end.
```

Para el caso base $n = 0$, lo cual significa que nuestra imagen tiene tamaño $2^0 \times 2^0 = 1 \times 1$. De ello deriva que la única coordenada contenida es $(0, 0)$; la región es evidentemente uniforme, por lo que se devuelve el color asociado al píxel correspondiente.

Si las proporciones de la imagen son mayores, el algoritmo divide la imagen y evalúa recursivamente sus cuatro cuadrantes. Sólo si los cuatro cuadrantes resultan ser homogéneos, y además, coinciden exactamente en la misma representación de color, la imagen completa se considera homogénea.

Sabemos que definir una función respetando la firma que se le fue asociada no necesariamente es prueba de que dicha función realice lo que se espera, se requiere demostrar que esto sucede. Por ello, es necesario enunciar el teorema `es_homogenea_correcta`, el cual asegura que si la función `es_homogenea` detecta un color uniforme, cualquier coordenada válida dentro de esa imagen devolverá dicho color [5] :

```
Theorem es_homogenea_correcta : ∀ n img color,
  es_homogenea n img = Some color →
  (∀ x y Hx Hy, img x y Hx Hy = color).
Proof.
  (* Demostración omitida en el documento por extensión de la prueba *)
Admitted.
```

La idea central de la demostración procede por inducción sobre el tamaño de la imagen. El caso base se resuelve de manera directa a partir del lema de Mapeo de píxeles, ya que la única coordenada posible es el origen. Mientras que en el paso inductivo, se realiza un análisis por casos sobre las coordenadas globales (x, y); dependiendo de la región a la que pertenezca la coordenada, y descartando los casos lógicamente absurdos, se aplica la hipótesis de inducción del cuadrante requerido y las metas obtenidas se reducen nuevamente al lema de Mapeo de píxeles.

Una vez que hemos, resuelto la necesidad de determinar cuando se cumple que una región sea homogenea, es posible definir la función encargada de construir la estructura jerárquica del árbol: `construye_region_qt`.

```
Fixpoint construye_region_qt (n : nat) : imagen n → region_qt :=
  match n with
  | 0 ⇒
     fun img ⇒
       Leaf (img 0 0 (Nat.lt_0_1) (Nat.lt_0_1))
  | S n' ⇒
     fun img ⇒
       match es_homogenea (S n') img with
       | Some color ⇒ Leaf color
```

[5] Derivado de todos los escenarios posibles que genera la división del plano y la representación de color para cada uno de ellos, el desarrollo formal de esta demostración es notablemente extenso. Por lo tanto, para consultar la construcción de la prueba, se debe revisar el código fuente que acompaña a este trabajo.

```
        | None ⇒
            Node (construye_region_qt n' (cuadrante_NW img))
                 (construye_region_qt n' (cuadrante_NE img))
                 (construye_region_qt n' (cuadrante_SW img))
                 (construye_region_qt n' (cuadrante_SE img))
        end
  end.
```

Como pudo haberse previsto, la construcción se mantiene simple tras haber delegado tareas a diferentes secciones de la formalización.
Si se determina que una imagen tiene color uniforme, la división se detiene representando todo el espacio en un único nodo hoja (`Leaf color`). En caso contrario, el espacio se divide generando un nodo gris (`Node`), y de manera recursiva, los hijos de este nodo continúan construyéndose, lo cual permite que se preserve la información de manera ordenada respecto a los cuadrantes de la imagen original.

4.3. Recuperación de la Imagen

Tras haber formalizado la construcción del árbol a partir de la división en subcuadrantes, es coherente pensar en la formalización del proceso inverso: recuperar la imagen almacenada en el RQT. Este proceso resulta de realizar un recorrido por profundidad en el árbol.

Notemos que al recorrer el árbol, eventualmente se visita un nodo hoja, que contiene, por definición, un color único que representa a toda una región. De manera que es posible representar la recuperación de la región homogénea utilizando una función constante:

```
Definition generar_imagen_homogenea (n : nat) (color : pixel) :
    imagen n :=
        fun x y Hx Hy ⇒ color.
```

Haciendo uso de esta definición, es posible modelar la función `recorre_region_qt`:

```
Fixpoint recorre_region_qt (n : nat) (qt : region_qt) : imagen n :=
  match qt with
  | Leaf color ⇒ generar_imagen_homogenea n color
  | Node nw ne sw se ⇒
      match n with
      | 0 ⇒ generar_imagen_homogenea 0 false
      | S n' ⇒
          ensamblar (recorre_region_qt n' nw)
                    (recorre_region_qt n' ne)
                    (recorre_region_qt n' sw)
                    (recorre_region_qt n' se)
      end
  end.
```

Es conveniente mencionar que la función procede por casos sobre la estructura del árbol:

- **Constructor `Leaf`.** Si encuentra una hoja, delega la creación de la región a la función homogénea previamente descrita.
- **Construtor `Node`.** Si se encuentra con un nodo interno (`Node`), el algoritmo se ve obligado a proceder nuevamente por casos, puesto que $n \in \mathbb{N}$.

- $n = 0$

  El píxel se define como mínima representación de color, por lo que no se puede dividir y este caso es imposible. Sin embargo, para preservar la totalidad de la función, se devuelve un valor por defecto.

- $n > 0$

  Cuando este caso ocurre, el algoritmo recupera de manera recursiva la subimagen de cada uno de los cuatro cuadrantes aplicando la función `ensamblar` para unir cada región dividida en una única imagen.

4.4. Función Identidad

Antes de presentar el teorema más importante de esta sección, es necesario indicarle al asistente de prueba que no es necesario extender la definición de la función `es_homogenea`:

```
Opaque es_homogenea.
```

Esto es consecuencia de que permitir su expansión alargaría algunas de las demostraciones posteriores. Al no utilizar la instrucción anterior, Rocq intentaría evaluar todos los casos posibles para los cuales está definida la función, lo cual es innecesario tras haber comprobado su correctitud en el teorema `es_homogenea_correcta`.

A partir de lo anterior, es posible presentar el resultado con el cual concluye la formalización de la estructura. El siguiente teorema verifica que la composición de funciones: `construye_region_qt` seguida de `recorre_region_qt` resulta en la función identidad.

```
Theorem identidad_region_qt : ∀ (n : nat) (img : imagen n),
  recorre_region_qt n (construye_region_qt n img) = img.
```

La demostración de este teorema procede por inducción sobre el valor del exponente, n, que dicta las proporciones de la imagen. A continuación, se detallan los pasos más significativos de la demostración [6] .

Para el caso base ($n = 0$), la imagen se reduce a un solo píxel.
Al aplicar la composición de funciones, el resultado es una función creada a partir de la definición de `generar_imagen_homogenea`. Utilizando el principio de extensionalidad funcional, se demuestra que la función constante es exactamente igual a la función que define la imagen original mediante el lema correspondiente a Mapeo de píxeles, pues la única coordenada válida para ambos lados de la ecuación es exactamente el origen $(0, 0)$.

Para el paso inductivo ($n > 0$), la imagen posee proporciones suficientes para ser dividida en cuadrantes. El análisis se sigue del resultado obtenido tras evaluar la homogeneidad de la imagen completa:

```
  - simpl construye_region_qt.
    destruct (es_homogenea (S n') img) as [color |] eqn:Hom.
```

Si la evaluación determina que los cuadrantes contienen regiones del mismo color, usamos el principio de extensionalidad funcional para mostrar que la función obtenida es exactamente igual a la función que define la imagen original. Esta demostración se cierra al aplicar

[6] Para consultar el desarrollo exacto de la prueba y la resolución completa de las metas intermedias, es necesario revisar el código fuente que acompaña a este trabajo.

el teorema `es_homogenea_correcta`, verificando que si una imagen cumple la propiedad mencionada, cualquier coordenada válida tendrá el color que representa a dicha hoja.

Finalmente, si la imagen se compone de cuadrantes con distintos colores (es decir, la función `es_homogenea` devuelve `None`), el recorrido por profundidad se aplicará recursivamente sobre los nodos equivalentes a las regiones Noroeste, Noreste, Suroeste y Sureste:

```
    + simpl recorre_region_qt.
      rewrite IHn'; rewrite IHn';  rewrite IHn';  rewrite IHn'.
      apply teorema_ensamblaje.
Qed.
```

Por hipótesis de inducción, sabemos que cada uno de estos recorridos recuperan las subimagenes locales de manera correcta, por lo cual, la prueba concluye tras aplicar el teorema de ensamblaje.

El proceso conjunto de toda esta sección demuestra, bajo la correspondencia de Curry-Howard, que el Region Quadtree es una estructura bien formada, pues la composición de sus funciones garantiza que la información de entrada conserva exactamente los mismos datos que la información que provee en su salida.

5. Invariantes RQT

En esta sección, nuestro objetivo es formalizar dos invariantes que demuestran la correctud y la propiedad óptima del Region Quadtree. Para ello, verificamos la imposibilidad de que exista un nodo gris, padre de 4 hijos que representen una región uniforme asociada al mismo color; adicionalmente, se establecerá la cota máxima de altura para cualquier RQT.

5.1. Representación Mínima

Para que la compresión de información espacial sea óptima, es necesario asegurar que el algoritmo no genere divisiones redundantes. El objetivo para comprobar lo anterior, es verificar que ningún nodo interno del árbol se utilice para representar una región que es, en su totalidad, homogénea.

5.1.1. Predicado de Minimalidad

Definimos entonces, el predicado `es_minimo`:

```
Inductive es_minimo : region_qt → Prop :=
  | Min_Leaf : ∀ c, es_minimo (Leaf c)
  | Min_Node : ∀ nw ne sw se,
      es_minimo nw →
      es_minimo ne →
      es_minimo sw →
      es_minimo se →
      ~ (∃ c, nw = Leaf c /\ ne = Leaf c /\ sw = Leaf c /\ se = Leaf c) →
      es_minimo (Node nw ne sw se).
```

El constructor `Min_Leaf` establece como caso base que el color de una región representado por una hoja es, por definición, la expresión mínima posible para esa área. Por otro lado, si se tiene un nodo intermedio, el constructor `Min_Node` determina que dicha partición es mínima únicamente si cumple las siguientes condiciones:

1. Los cuatro subárboles que representa cumplen con el predicado de minimalidad.
2. Se rechaza la posibilidad de que los cuatro hijos sean hojas del mismo color. Es decir, al menos una de las subregiones debe diferir del resto.

5.1.2. Lemas Auxiliares de Construcción

Antes de abordar el teorema principal, requerimos demostrar dos propiedades sobre el comportamiento de nuestras funciones. La primera se establece como un lema, el cual, garantiza que si el algoritmo de construcción devuelve una hoja (`Leaf c`), entonces la función `es_homogenea` identificó la región como homogénea (`Some c`).

```
Lemma construye_leaf_homogenea : ∀ n img c,
  construye_region_qt n img = Leaf c → es_homogenea n img = Some c.
Proof.
  intros n img c.
  destruct n as [| n'].
  - intros H.
    simpl in H.
    injection H as H_color.
    rewrite <- H_color.
    reflexivity.
  - intros H.
    simpl in H.
    destruct (es_homogenea (S n') img) eqn:Hom.
    + injection H as Hc.
      rewrite Hc.
      reflexivity.
    + discriminate H.
Qed.
```

La demostración procede por casos sobre el valor del exponente, n, que dicta las proporciones de la imagen. Cuando $n = 0$, la imagen representa un solo píxel, el cual, por definición, es homogéneo, de ello se concluye directamente la igualdad. Para $n > 0$, el análisis recae sobre el resultado de `es_homogenea`. Destacando el caso donde se obtiene el color de la región con `Some c`, pues cuando la función devuelve `None`, el constructor jamás podrá cumplir la construcción a partir de `Leaf`.

El segundo lema auxiliar requiere comprobar la dirección inversa de la evaluación. Para ello, permitimos temporalmente que la definición de la función `es_homogenea` vuelva a expandirse:

```
Transparent es_homogenea.
```

El siguiente lema establece que, si los cuadrantes de una imagen arrojan exactamente el mismo color, `c`, entonces la función `es_homogenea` devolverá `Some c` para la imagen original.

```
Lemma composicion_homogenea : ∀ n img c,
  es_homogenea n (cuadrante_NW img) = Some c →
  es_homogenea n (cuadrante_NE img) = Some c →
  es_homogenea n (cuadrante_SW img) = Some c →
  es_homogenea n (cuadrante_SE img) = Some c →
  es_homogenea (S n) img = Some c.
Proof.
  intros n img c Hnw Hne Hsw Hse.
  simpl.
  rewrite Hnw; rewrite Hne; rewrite Hsw; rewrite Hse.
  destruct c.
  + reflexivity. (* Some true = Some true *)
```

```
  + reflexivity. (* Some false = Some false *)
Qed.
```

La demostración simplifica la aplicación de la función y resuelve las llamadas recursivas reescribiendo las hipótesis de los cuadrantes. Dado que la representación del color utiliza valores booleanos, el análisis por casos sobre el color representativo genera igualdades inmediatas, finalizando así la prueba.

5.1.3. Region Quadtree es Mínimo

Tras demostrar que el comportamiento de la función `es_homogenea` es correcto, volvemos a ocultar su estructura interna para las siguientes demostraciones:

```
Opaque es_homogenea.
```

Utilizando el predicado y los lemas auxiliares establecidos, es posible formalizar la invariante de nuestro interés. Enunciamos la invariante como un teorema, el cual establece que para cualquier imagen válida, el algoritmo de construcción siempre genera un árbol mínimo.

```
Theorem construccion_es_minima : ∀ n img,
  es_minimo (construye_region_qt n img).
```

La demostración procede mediante inducción estructural sobre n. En el caso base, la imagen es un píxel. Como cualquier píxel genera una hoja, se satisface de manera directa el predicado de minimalidad.

```
  - intros img.
    simpl construye_region_qt.
    destruct (es_homogenea (S n') img) as [color |] eqn:Hom.
    + apply Min_Leaf.
```

Para el caso inductivo, es posile dividir la imagen posee cuatro cuadrantes, por lo cual, la demostración se divide en casos según el análisis de homogeneidad.

Si los cuadrantes conforman una región uniforme, el algoritmo detiene la ramificación y genera una hoja, la cual es mínima por definición y se aplica nuevamente el hecho `Min_Leaf`.

Cuando la imagen contiene colores distintos, el algoritmo genera un nodo intermedio, y para verificar su minimalidad, aplicamos la hipótesis de inducción sobre sus cuatro hijos:

```
    + apply Min_Node.
      * apply IHn'.
      * apply IHn'.
      * apply IHn'.
      * apply IHn'.
```

Resta asegurar que al menos uno de los cuadrantes representa un color distinto del resto. Procedemos por contradicción: Suponemos que los cuatro subárboles son hojas del mismo color, `c`. Aplicando el lema `construye_leaf_homogenea`, afirmamos que cada cuadrante devolvió `Some c`.

Por el lema de composición homogénea, tenemos que la función `es_homogenea` evaluada sobre la imagen completa debió devolver `Some c`. Lo cual es una contradicción, pues el caso donde nos encontramos parte de que la evaluación inicial devolvió `None`.

```
        assert (Contra : es_homogenea (S n') img = Some c).
        { exact (composicion_homogenea n' img c Hnw Hne Hsw Hse). }
```

```
        rewrite Hom in Contra.
        discriminate Contra.
Qed.
```

Notemos que la táctica `discriminate` reconoce esta contradicción y resuelve la meta instantáneamente, completando así la demostración del invariante correspondiente a la minimalidad de la estructura.

5.2. Altura Máxima

Como segunda propiedad estructural, analizaremos el crecimiento vertical del árbol. Demostrando que el proceso de ramificación de una imagen con tamaño $2^n \times 2^n$ está estrictamente acotado por el exponente n.

5.2.1. Definición Altura

Para comprobar la cota propuesta, definimos la función `altura_qt`, la cual calcula la longitud del camino más largo desde la raíz hasta la hoja más profunda de un RQT.

```
Fixpoint altura_qt (qt : region_qt) : nat :=
  match qt with
  | Leaf _ ⇒ 0
  | Node nw ne sw se ⇒
      S (Nat.max (Nat.max (altura_qt nw) (altura_qt ne))
                 (Nat.max (altura_qt sw) (altura_qt se)))
  end.
```

5.2.2. Teorema de Altura Máxima

Con la función estándar establecida, podemos enunciar el teorema que modela nuestro resultado: para cualquier imagen con proporciones $2^n \times 2^n$ (cuyo número total de píxeles es 4^n), la altura máxima de un RQT generado será, a lo más, $\log_4(4^n) = n$.

```
Theorem altura_max : ∀ n img,
  altura_qt (construye_region_qt n img) ≤ n.
```

La prueba procede por inducción estructural sobre n. El caso base se obtiene cuando la imagen se reduce a un solo píxel ($n = 0$), por definición de `construye_region_qt` esta region se almacena en una una hoja, y la función de altura asigna un 0 a su constructor. Dado que la meta se reduce a $0 \leq 0$, la táctica de aritmética lineal resuelve la desigualdad de inmediato, cumpliéndose para el caso base.

Para el paso inductivo ($n > 0$), por lo cual la imagen tiene la capacidad de dividirse en cuatro cuadrantes. Esto despliega dos casos que dependen del resultado de la función `es_homogenea`. En el primer escenario, los cuadrantes contienen regiones homogéneas del mismo color y, por tanto, se representan mediante una sola hoja. La altura de un nodo hoja es 0, por lo que basta demostrar que $0 \leq S\ n'$, lo cual es verdadero para cualquier número natural n'.

```
  - intros img.
    simpl construye_region_qt.
    destruct (es_homogenea (S n') img) as [color |] eqn:Hom.
    + simpl altura_qt; lia.
```

El segundo subcaso ocurre cuando la imagen se compone de cuadrantes con variación de color. Notemos que la representación se da con el constructor `Node` y los cuatro subárboles

correspondientes. Por hipótesis de inducción, sabemos que la propiedad `altura_max` se cumple para cada uno de estos cuadrantes, por lo que podemos afirmar que la altura de cada subárbol es, a lo más, n'.

```
    + simpl altura_qt.
      assert
        (Hnw: altura_qt (construye_region_qt n' (cuadrante_NW img)) ≤ n')
      { exact (IHn' (cuadrante_NW img)). }
      assert
        (Hne: altura_qt (construye_region_qt n' (cuadrante_NE img)) ≤ n')
      { exact (IHn' (cuadrante_NE img)). }
      assert
        (Hsw: altura_qt (construye_region_qt n' (cuadrante_SW img)) ≤ n')
      { exact (IHn' (cuadrante_SW img)). }
      assert
        (Hse: altura_qt (construye_region_qt n' (cuadrante_SE img)) ≤ n')
      { exact (IHn' (cuadrante_SE img)). }
      lia.
Qed.
```

Derivado de estas afirmaciones, podemos asegurar que la altura del nodo padre será, a lo más, $n = S\ n'$, cerrando así la demostración.

6. Template Quadtree

El análisis realizado en las secciones anteriores permite la formalización de homogeneidad del color sobre una región a partir de un modelo de geometria computacional discreta basado en la traducción algebraica, los tipos dependientes y la correspondencia Curry-Howard. Sin embargo, en el procesamiento de imágenes, las representaciones binarias suelen encontrar un límite práctico al enfrentarse a patrones más elaborados en las imágenes.

Para superar esta limitante se desarrolló el *Template Quadtree* [3]. Esta variante expande la capacidad de compresión al reconocer patrones espaciales predefinidos, permitiendo comprimir imágenes de tamaño $2^n \times 2^n$ en un solo nodo hoja que almacena, no una matriz de píxeles, sino un simple identificador de plantilla.

A continuación, expondremos cómo el modelo resultante de la traducción al dominio algebraico permite extender nuestro modelo para formalizar esta nueva arquitectura resulta un proceso directo y natural.

6.1. Abstracción del Diccionario de Plantillas

La transición de la homogeneidad de color a la coincidencia de patrones espaciales requiere un algoritmo de búsqueda., sin embargo, para evitar codificar estáticamente todas las configuraciones posibles de plantillas en el asistente de prueba, utilizaremos parámetros y axiomas para abstraer el diccionario donde se espera, las plantillas estén almacenadas:

```
Parameter TemplateID : Type.
Parameter bd_plantillas : ∀ n, TemplateID → imagen n.

Parameter busca_plantilla : ∀ n, imagen n → option TemplateID.

Axiom busca_plantilla_correcta : ∀ n img id,
  busca_plantilla n img = Some id → bd_plantillas n id = img.
```

El axioma `busca_plantilla_correcta` será la suposición formal que valida que si una imagen de tamaño 2^n coincide con una plantilla almacenada en la base de datos, el diccionario devolverá exactamente la imagen correspondiente a la imagen indicada al realizar la consulta.

6.2. Definición en Rocq y Construcción

Para formalizar el TQT, adaptamos la definición para que sea capaz de distinguir cuando una hoja almacena un píxel unitario, cuando se almacena la referencia a una plantilla del diccionario y cuando ocurre la division recursiva en cuadrantes:

```
Inductive TemplateQuadtree : Type :=
  | Leaf_pixel : pixel → TemplateQuadtree
  | Leaf_plantilla : TemplateID → TemplateQuadtree
  | Node : TemplateQuadtree → TemplateQuadtree → TemplateQuadtree →
   TemplateQuadtree → TemplateQuadtree.
```

Utilizando esta estructura, el algoritmo de construcción resulta inmediato, tal y como ocurrió con RQT: Si la subimagen actual coincide con una plantilla, se utiliza el constructor `Leaf_plantilla`, representando todo el bloque en una sola referencia. Cuando esto no ocurre, se subdivide a los cuadrantes ya conocidos mediante el constructor `Node`, y, si la subdivisión se reduce hasta llegar a un pixel, lo establecemos dentro de un nodo hoja usando `Leaf_pixel`:

```
Fixpoint construye_template (n : nat) : imagen n → TemplateQuadtree :=
  match n with
  | 0 ⇒
      fun img ⇒ Leaf_pixel (img 0 0 (Nat.lt_0_1) (Nat.lt_0_1))
  | S n' ⇒
      fun img ⇒
        match busca_plantilla (S n') img with
        | Some id ⇒ Leaf_plantilla id
        | None ⇒
            Node (construye_template n' (cuadrante_NW img))
                 (construye_template n' (cuadrante_NE img))
                 (construye_template n' (cuadrante_SW img))
                 (construye_template n' (cuadrante_SE img))
        end
  end.
```

6.3. Recuperación de la Imagen y Función Identidad

Notemos que el recorrido para esta variante se realiza usando la misma lógica que el presentado para RQT, por lo cual, la demostración de identidad geométrica reutiliza directamente el Lema de Ensamblaje desarrollado para el Region Quadtree. La única variación radica en que la exactitud del contenido de las hojas se delega al axioma del diccionario, demostrando que la formalización base es capaz de expandirse a otras variantes destinadas a representación de imágenes con píxeles.

El teorema de identidad, se enuncia de la siguiente manera [7] :

```
Theorem isomorfismo_template : ∀ (n : nat) (img : imagen n),
  recupera_template n (construye_template n img) = img.
```

[7] La implementacion del recorrido y la explicación de su demostración son redundantes tras haber detallado su comportamientopara RQT, por lo que el contenido aparece únicamente en el codigo fuente que acompaña este trabajo.

6.4. Invariante Árbol Mínimo

En esta variante, un árbol es considerado mínimo no solo por carecer de hojas de color idéntico, sino porque el algoritmo jamás ignora la existencia de una plantilla almacenada en el diccionario. El siguiente predicado modela formalmente que, si un nodo interno existe ,es consecuencia de que la búsqueda de plantilla devolvió `None`:

```
Inductive es_minimo_tqt : nat → TemplateQuadtree → Prop :=
  | Min_Leaf_pixel : ∀ p, es_minimo_tqt 0 (Leaf_pixel p)
  | Min_Leaf_plantilla: ∀ n id, es_minimo_tqt n (Leaf_plantilla id)
  | Min_Node : ∀ n nw ne sw se img,
      es_minimo_tqt n nw →
      es_minimo_tqt n ne →
      es_minimo_tqt n sw →
      es_minimo_tqt n se →
      recupera_template (S n) (Node nw ne sw se) = img →
      busca_plantilla (S n) img = None →
      es_minimo_tqt (S n) (Node nw ne sw se).
```

La cláusula principal del constructor `Min_Node` determina que si fue necesario ensamblar cuatro subárboles para recuperar una imagen, fue unicamente porque el mecanismo de búsqueda verificó la inexistencia de dicha plantilla en el nivel correspondiente.

Adicionalmente, sabemos que el proceso de subdivisión mantuvo exactamente la misma lógica que el presentado en RQT, por lo cual, la segunda invariante presentada en nuestra formalización base, se hereda, de manera que la altura máxima de cualquier Template Quadtree generado para una imagen de proporciones $2^n \times 2^n$ se mantendrá estrictamente limitada por el valor de n.

7. Conclusiones

A lo largo de este trabajo, hemos consolidado la formalización del algoritmo base para quadtrees destinados a la representación de imágenes generadas a través de píxeles. Al traducir este problema a un dominio algebraico que se apoya de la teoría de tipos dependientes y la correspondencia de Curry-Howard, fue posible automatizar su verificación en el asistente de pruebas Rocq. Logrando así demostrar que en las variantes desarrolladas, la preservación de datos ocurre, lo cual resulta indispensable cuando la estructura se utiliza en software crítico.

El desarrollo de este enfoque puso en evidencia que modelar el espacio a través de una traducción algebraica directa reduce significativamente la complejidad de las demostraciones. Al establecer estas bases, fue posible utilizar tácticas de resolución lineal, para resolver metas geométricas apoyándose exclusivamente en las propiedades de los números naturales.

La formalización desarrollada sobre la variante RQT indica que los nodos intermedios son necesarios únicamente para recuperar el contenido de las hojas en el orden establecido. Derivado de esto, el modelo resultó adecuado y eficiente para extender la formalización hacia estrategias de compresión por patrones, tal como se indicó para la siguiente variante, Template Quadtree.

En contraste a lo anterior, la extensión del enfoque analítico hacia representaciones de imágenes a partir de vectores presenta limitaciones evidentes. Variantes diseñadas a este tipo de archivos, como el *Line Quadtree* y el *Edge Quadtree* [3], no fundamentan su partición en la homogeneidad de píxeles, sino en la intersección de curvas continuas y fronteras geométricas arbitrarias. En tales escenarios, el modelo presentado resulta insuficiente, pues la traducción

al dominio algebraico se desenvuelve como un predicado que puede decidir si una propiedad se cumple o no en el dominio geométrico, sin embargo, en las representaciones vectoriales los nodos internos están obligados a almacenar información geométrica adicional para asegurar la recuperación de las figuras descritas originalmente.

Cuando el criterio de partición abandona la representación discreta y comienza a depender de intersecciones continuas, la evolución de esta investigación requiere modelarse sobre la topología de las imágenes de manera directa. Tal y como sugieren trabajos en el área [12], será indispensable utilizar modelos basados en hipermapas (modelos formales que describen como se conectan y relacionan los vértices, aristas y caras en una figura) para determinar con exactitud los invariantes que cumplen las estructuras geométricas pendientes a formalizar.

Referencias

[1] R. A. Finkel and J. L. Bentley, "Quad trees: A data structure for retrieval on composite keys," *Acta Informatica*, vol. 4, pp. 1–9, 1974.

[2] A. Klinger and C. R. Dyer, "Experiments in picture representation using regular decomposition," *Computer Graphics and Image Processing*, vol. 5, no. 1, pp. 68–105, 1976.

[3] S. S. Iyengar, V. K. Vaishnavi, and S. Gunasekaran, "Image data structures," in *Handbook of Data Structures and Applications* (D. P. Mehta and S. Sahni, eds.), ch. 58 Image Data Structures, pp. 903–916, Boca Raton, FL: CRC Press, Taylor & Francis Group, 2 ed., 2018.

[4] D. Yang, P. Vicol, X. Qi, R. Liao, and X. Zhang, "Qdm: Quadtree-based region-adaptive sparse diffusion models for efficient image super-resolution," 2025.

[5] S. Dua, N. Kandiraju, and P. Chowriappa, "Region quad-tree decomposition based edge detection for medical images," *The Open Medical Informatics Journal*, vol. 4, pp. 50–57, 2010.

[6] C. L. Marcus, R. Zanetti, and T. P. Setterfield, "Landing site mapping and selection with a quadtree map using planar elements," *Journal of Guidance, Control, and Dynamics*, vol. 47, pp. 1283–1297, 2024.

[7] M. Soto Romero, *Programación funcional en Rocq.* Facultad de Ciencias, UNAM, feb 2026. Nota de Clase.

[8] B. C. Pierce, A. A. de Amorim, C. Casinghino, M. Gaboardi, M. Greenberg, C. Hriţcu, V. Sjöberg, and B. Yorgey, *Logical Foundations*, vol. 1 of *Software Foundations.* Electronic textbook, 2026. Version 7.0, `http://softwarefoundations.cis.upenn.edu`.

[9] L. D. C. González Huesca, K. Ramírez Pulido, and K. S. Romero Estrada, *Lenguajes con tipificado dependiente.* Facultad de Ciencias, UNAM, 2026.

[10] M. Beeson, "Proof and computation in geometry," in *Automated deduction in geometry*, vol. 7993 of *Lecture Notes in Comput. Sci.*, pp. 1–30, Springer, Heidelberg, 2013.

[11] M. Soto Romero, *Lógica proposicional y de primer orden.* Facultad de Ciencias, UNAM, mar 2026. Nota de Clase.

[12] J.-F. Dufourd, "Design and formal proof of a new optimal image segmentation program with hypermaps," *Pattern Recognition*, vol. 40, no. 10, pp. 2974–2993, 2007.